Trees in Data Structures

1. General Tree

- A **Tree** is a non-linear hierarchical data structure consisting of nodes connected by edges.
- It represents hierarchical relationships between elements.
- The topmost node is called the **Root**.
- A node can have zero or more child nodes.
- A node without children is called a **Leaf Node**.
- A node having at least one child is called an **Internal Node**.
- The connection between two nodes is called an **Edge**.
- A node's parent is the immediate predecessor of that node.
- Nodes having the same parent are called **Siblings**.
- **Degree of a node:** Number of children of that node.
- **Degree of a tree:** Maximum degree of any node in the tree.
- **Depth of a node:** Number of edges from root to that node.
- **Height of a node:** Number of edges in the longest path from that node to a leaf.
- **Height of a tree:** Height of the root node.

Important Properties

- A tree with `n` nodes has exactly `n - 1` edges.
- There is exactly one path between any two nodes in a tree.
- A tree is a connected and acyclic data structure.
- Root node has no parent.
- Leaf nodes have no children.

SUNBEAM

---

2. Binary Tree

- A **Binary Tree** is a tree in which each node can have at most two children.
- The two children are called:
  - Left Child
  - Right Child
- A binary tree can have zero, one or two children per node.
- Each node generally contains data and references to left and right children.

Java Node Structure

```
class Node {
    int data;
    Node left;
    Node right;
}
```

Types of Binary Trees

1. Full Binary Tree (Strict/Proper Binary Tree)

- Every node has either zero or exactly two children.
- No node has only one child.

2. Complete Binary Tree

- All levels are completely filled except possibly the last level.
- The last level is filled from left to right.
- Used in implementing Binary Heap.

3. Perfect Binary Tree

- All internal nodes have exactly two children.
- All leaf nodes are at the same level.

SUNBEAM

- Every level is completely filled.

4. Balanced Binary Tree

- The heights of left and right subtrees are balanced according to a defined condition.
- For AVL trees, the height difference between left and right subtrees must not exceed 1.

5. Skewed Binary Tree

- Every node has only one child.
- Types:
  - Left Skewed: Every node has only a left child.
  - Right Skewed: Every node has only a right child.
- Resembles a linked list.
- Height can become O(n).

Important Binary Tree Properties

- Maximum number of nodes at level `L` (root at level 0): `2^L`.
- Maximum number of nodes in a binary tree of height `h` (height measured in edges): `2^(h+1) - 1`.
- Maximum height of a binary tree with `n` nodes: `n - 1`.
- Minimum possible height for `n` nodes: `ceil(log2(n + 1)) - 1`.
- Maximum number of leaf nodes in a binary tree of height `h`: `2^h`.
- In a full binary tree:
  - If `I` = number of internal nodes and `L` = number of leaf nodes,
  - `L = I + 1`.
  - Total nodes = `2I + 1`.

SUNBEAM

---

3. Binary Search Tree (BST)

- A **Binary Search Tree** is a binary tree that follows a specific ordering property.

- For every node:
  - All keys in the left subtree are smaller than the node's key.
  - All keys in the right subtree are greater than the node's key.
- Both left and right subtrees must also follow the BST property.
- Duplicate values are generally not allowed in a basic BST; handling duplicates depends on implementation.

Example

```
BST:

       50
     /   \
    30    70
   / \    / \
  20 40  60 80
```

BST Operations

Operation	Average Case	Worst Case
Search	O(log n)	O(n)
Insertion	O(log n)	O(n)
Deletion	O(log n)	O(n)

- Average complexity assumes a reasonably balanced tree.
- Worst case occurs when BST becomes skewed.

BST Searching

- Compare the key with the current node.
- If key equals current node, element is found.

SUNBEAM

- If key is smaller, search the left subtree.
- If key is greater, search the right subtree.
- Continue until the key is found or a null reference is reached.

BST Insertion

- Start from the root.
- Compare the new key with the current node.
- Move left if the key is smaller.
- Move right if the key is greater.
- Insert the new node at the first available null position.

BST Deletion

Three cases:

1. **Leaf Node**
    - Directly remove the node.
2. **Node with One Child**
    - Replace the node with its only child.
3. **Node with Two Children**
    - Replace the node with its Inorder Successor or Inorder Predecessor.
    - Inorder Successor: Minimum value in the right subtree.
    - Inorder Predecessor: Maximum value in the left subtree.
    - Delete the replacement node from its original position.

BST Traversals

- **Inorder (Left, Root, Right):** Produces sorted order in ascending order for a BST with distinct keys.
- **Preorder (Root, Left, Right):** Root is visited first.

SUNBEAM

- **Postorder (Left, Right, Root):** Root is visited last.
- **Level Order:** Visits nodes level by level using a Queue.

Important MCQ Points

- Inorder traversal of BST gives sorted ascending order for distinct keys.
- Searching in a balanced BST takes O(log n).
- Searching in a skewed BST takes O(n).
- Minimum element is the leftmost node.
- Maximum element is the rightmost node.
- Inorder successor is the next greater element.
- Inorder predecessor is the next smaller element.

---

4. Balanced BST - AVL Tree

- AVL Tree is a self-balancing Binary Search Tree.
- It was introduced by **Adelson-Velsky and Landis**.
- It maintains the BST ordering property.
- After insertion or deletion, the tree is rebalanced if required.
- The balance of a node is maintained using its Balance Factor.

Balance Factor (BF)

```
Balance Factor = Height(Left Subtree) - Height(Right Subtree)
```

- For every node in an AVL tree, the balance factor must be:
  - -1
  - 0
  - +1

SUNBEAM

- If balance factor becomes less than -1 or greater than +1, the node is unbalanced.
- Rotations are used to restore balance.

AVL Tree Complexity

Operation	Time Complexity
Search	O(log n)
Insertion	O(log n)
Deletion	O(log n)
Rotation	O(1)

Advantages

- Maintains height balance automatically.
- Search operation is efficient.
- Prevents the tree from becoming skewed.
- Guarantees O(log n) search, insertion and deletion.

Disadvantages

- Requires additional balance factor/height information.
- Insertion and deletion are more complex than ordinary BST.
- Rotations require additional operations.

SUNBEAM

---

5. AVL Tree Rotations

- Rotation is a restructuring operation used to restore AVL tree balance.
- It maintains the BST ordering property.
- Four imbalance cases are handled using rotations:

1. LL Case
2. RR Case
3. LR Case
4. RL Case

5.1 Right Rotation

- Right rotation is used to fix the **LL imbalance**.
- The left child becomes the new root of the rotated subtree.
- The original root becomes the right child of the new root.
- The right subtree of the new root becomes the left subtree of the original root.

Before Rotation

```
      z
     /
    y
   / \
  T1  T2
```

After Right Rotation

```
      y
     / \
    T1  z
       /
      T2
```

Java Code

SUNBEAM

```
Node rightRotate(Node z) {
    Node y = z.left;
    Node T2 = y.right;

    y.right = z;
    z.left = T2;

    updateHeight(z);
    updateHeight(y);

    return y;
}
```

5.2 Left Rotation

- Left rotation is used to fix the **RR imbalance**.
- The right child becomes the new root of the rotated subtree.
- The original root becomes the left child of the new root.
- The left subtree of the new root becomes the right subtree of the original root.

Before Rotation

```
    z
     \
      y
     / \
    T2  T3
```

After Left Rotation

SUNBEAM

```
        y
       / \
      z   T3
       \
        T2
```

Java Code

```
Node leftRotate(Node z) {
    Node y = z.right;
    Node T2 = y.left;

    y.left = z;
    z.right = T2;

    updateHeight(z);
    updateHeight(y);

    return y;
}
```

Height Update in Java

```
int height(Node node) {
    return node == null ? 0 : node.height;
}

void updateHeight(Node node) {
    node.height = 1 + Math.max(
        height(node.left),
```

SUNBEAM

```
            height(node.right)
        );
    }
```

- The above code assumes leaf nodes have height 1 and null nodes have height 0.
- Height must be updated from the lower node to the new root after rotation.

---

6. AVL Imbalance Cases

6.1 LL Case (Left-Left)

- Occurs when a node becomes unbalanced due to insertion in the left subtree of its left child.
- Balance Factor of the unbalanced node is +2.
- Solution: **Single Right Rotation**.

Example: Insert 30, 20, 10.

Before Rotation

```
     30
    /
   20
  /
 10
```

After Right Rotation

```
     20
    /  \
```

SUNBEAM

```
  10   30
```

6.2 RR Case (Right-Right)

- Occurs when a node becomes unbalanced due to insertion in the right subtree of its right child.
- Balance Factor of the unbalanced node is -2.
- Solution: **Single Left Rotation**.

Example: Insert 10, 20, 30.

Before Rotation

```
10
  \
   20
     \
      30
```

After Left Rotation

```
    20
   /  \
  10   30
```

SUNBEAM

6.3 LR Case (Left-Right)

- Occurs when a node becomes unbalanced due to insertion in the right subtree of its left child.
- Balance Factor of the unbalanced node is +2.
- Solution:

1. Left rotation on the left child.
2. Right rotation on the unbalanced node.

Example: Insert 30, 10, 20.

Before Rotation

```
    30
   /
  10
    \
     20
```

Step 1: Left Rotation on 10

```
    30
   /
  20
 /
10
```

Step 2: Right Rotation on 30

```
    20
   /  \
  10   30
```

6.4 RL Case (Right-Left)

SUNBEAM

- Occurs when a node becomes unbalanced due to insertion in the left subtree of its right child.
- Balance Factor of the unbalanced node is -2.
- Solution:
    1. Right rotation on the right child.
    2. Left rotation on the unbalanced node.

Example: Insert 10, 30, 20.

Before Rotation

```
10
  \
   30
  /
 20
```

Step 1: Right Rotation on 30

```
10
  \
   20
     \
      30
```

Step 2: Left Rotation on 10

```
    20
   /  \
  10   30
```

SUNBEAM

7. Difference Between AVL Imbalance Cases

Case	Insertion Position	Balance Factor	Solution
LL	Left subtree of left child	+2	Right Rotation
RR	Right subtree of right child	-2	Left Rotation
LR	Right subtree of left child	+2	Left Rotation + Right Rotation
RL	Left subtree of right child	-2	Right Rotation + Left Rotation

Easy Identification

- LL → Left Left → Right Rotation.
- RR → Right Right → Left Rotation.
- LR → Left Right → Left Rotation followed by Right Rotation.
- RL → Right Left → Right Rotation followed by Left Rotation.

Important MCQ Points

- LL and RR are single rotation cases.
- LR and RL are double rotation cases.
- Right rotation fixes LL imbalance.
- Left rotation fixes RR imbalance.
- LR requires Left Rotation followed by Right Rotation.
- RL requires Right Rotation followed by Left Rotation.
- AVL Balance Factor is calculated as Left Height minus Right Height.
- Valid AVL Balance Factors are -1, 0 and +1.
- AVL tree maintains O(log n) height.

SUNBEAM

8. Applications of Trees

- **File Systems:** Represent directories and files in a hierarchical structure.
- **Database Indexing:** B-Trees and B+ Trees are used for efficient searching and indexing.
- **Searching:** BST and AVL trees support efficient search operations.
- **Priority Queues:** Binary Heaps are used to implement priority queues.
- **Expression Trees:** Represent arithmetic and logical expressions.
- **Compiler Design:** Abstract Syntax Trees (AST) represent program structure.
- **Decision Trees:** Used in machine learning and decision-making.
- **Networking:** Routing and hierarchical network structures.
- **HTML/XML:** DOM represents document elements in a tree structure.
- **Autocomplete:** Tries are used for prefix-based searching.
- **Compression:** Huffman Trees are used in data compression.
- **Operating Systems:** Hierarchical representation of processes and file systems.

9. Important MCQ Revision Points

- Tree is a non-linear hierarchical data structure.
- A tree with n nodes has n-1 edges.
- A binary tree has at most two children per node.
- A full binary tree has either zero or two children per node.
- A complete binary tree fills levels from left to right.
- A perfect binary tree has all levels completely filled.
- A skewed binary tree resembles a linked list.
- Inorder traversal of BST produces sorted ascending order for distinct keys.
- BST search complexity is O(log n) on average and O(n) in the worst case.
- AVL is a self-balancing BST.
- AVL was developed by Adelson-Velsky and Landis.
- AVL Balance Factor = Height(Left) - Height(Right).

SUNBEAM

- Valid AVL Balance Factors are -1, 0 and +1.
- LL → Right Rotation.
- RR → Left Rotation.
- LR → Left Rotation followed by Right Rotation.
- RL → Right Rotation followed by Left Rotation.
- AVL search, insertion and deletion take O(log n) time.
- B-Trees and B+ Trees are commonly used in database indexing.
- Binary Heap is commonly used to implement priority queues.

SUNBEAM